

MOVIE RECOMMENDATION SYSTEM

Machine Learning Project Report

Submitted by	Parita Dhanani
Branch	Computer Engineering
College	Shree Swaminarayan Institute of Technology
University	Gujarat Technological University (GTU)
Academic Year	2026
Enrollment Number	

Repository: DhananiParita/Movie-Recommendation-System
Prepared from the project repository and the supplied ML project report format.

Table of Contents

Placeholder for table of contents	0
-----------------------------------	---

Figures and Tables

- Figure 1. System architecture
- Table 1. Dataset and preprocessing summary
- Table 2. Core implementation components
- Table 3. Recommendation examples observed in the notebook

Abstract

This project presents a movie recommendation system that combines a content-based machine learning approach with a web application and live movie metadata. The core recommendation engine represents movie text as TF-IDF vectors and uses cosine similarity to identify titles that are close to a selected movie in the learned text space. During preprocessing, movie metadata is converted into a consolidated text field called tags, followed by lowercasing, punctuation removal, stop-word removal and lemmatization. The notebook processes 45,447 movie records and creates a 50,000-feature TF-IDF representation using unigrams and bigrams. The resulting vectorizer, matrix, movie dataframe and title-index mapping are saved as serialized files for reuse by the application.

The application layer is built with FastAPI and Streamlit. TMDB is used for movie search, posters, current movie details and genre-based discovery, while the local TF-IDF artifacts provide similarity-based recommendations. The repository therefore works as an end-to-end system rather than only a model notebook. Example notebook outputs show useful content relationships, such as Avatar returning Avatar 2 among its closest titles and The Dark Knight Rises returning other Batman-related films. Since the project has no labeled relevance target, classification metrics such as accuracy, precision, recall, F1-score and ROC-AUC are not appropriate. The current system is a practical baseline that can later be strengthened with collaborative filtering, better ranking, personalization and formal recommendation evaluation.

Chapter 1: Introduction

1.1 Background

Movie platforms contain thousands of titles, so the difficult part for a user is often not finding a movie, but deciding what to watch next. A recommendation system reduces that search effort by using information about movies to surface titles that are likely to feel relevant. This project approaches the problem from the movie-content side: if two movies have similar textual descriptions and metadata, they can be represented by similar numerical vectors and recommended to one another.

1.2 Problem Statement

The project addresses the problem of recommending movies from a large catalogue when a user selects or searches for a title. The system should provide useful similar-movie suggestions, while also giving the user practical information such as posters, release dates, genres and descriptions. A second requirement is usability: the recommendation logic should be available through an interface rather than remaining inside a notebook.

1.3 Motivation

The motivation is to connect a machine-learning technique with a usable application. Instead of stopping after generating a similarity matrix, the project packages the trained artifacts into an API and connects them to a Streamlit interface. TMDb integration makes the interface more useful because users can search movie metadata and view visual information.

1.4 Objectives

- Build a content-based movie recommendation engine using TF-IDF and cosine similarity.
- Prepare movie text so that descriptions and metadata can be compared consistently.
- Store trained TF-IDF resources so recommendations do not need to be retrained every time the application starts.
- Expose recommendation and movie-information functionality through a FastAPI backend.
- Create a Streamlit interface for search, movie details and recommendation browsing.
- Combine local similarity recommendations with TMDb genre-based discovery.

1.5 Scope

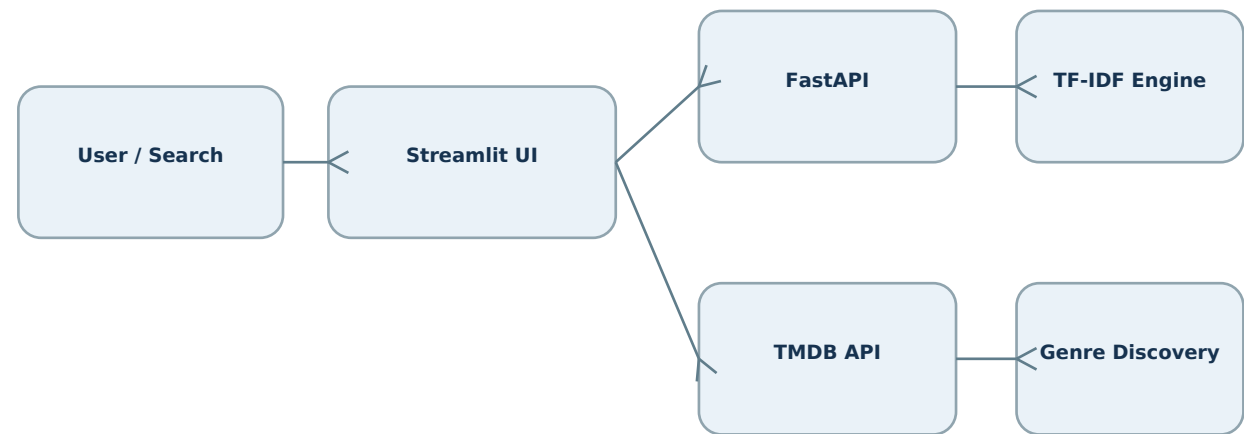
The present system focuses on content-based recommendations and metadata-driven discovery. It does not model individual user ratings, watch history or long-term preferences. The scope covers movie-to-movie similarity, title search, movie details, genre discovery and the supporting web application. Personalized collaborative filtering is outside the current implementation.

1.6 Report Organization

Chapter 1 introduces the problem and objectives. Chapter 2 explains the architecture and methodology. Chapter 3 describes the dataset and preprocessing. Chapter 4 discusses implementation, tools and saved artifacts. Chapter 5 analyzes observed results and evaluation limitations. Chapter 6 concludes the work and lists future improvements.

Chapter 2: Methodology / System Design

2.1 Overall Architecture



Local artifacts: df.pkl • indices.pkl • tfidf.pkl • tfidf_matrix.pkl
 TMDB: search • details • posters • genre discovery

Figure 1. System architecture of the implemented application.
 The system has two connected paths. The first is the local machine-learning path: the processed movie dataframe, title-to-index mapping, TF-IDF vectorizer and TF-IDF matrix are loaded by FastAPI and used to calculate similarity. The second is the external metadata path: the backend calls TMDB for search results, movie details, posters and genre discovery. Streamlit acts as the presentation layer and calls the FastAPI endpoints.

2.2 Content-Based Recommendation

The notebook creates a textual representation of each movie and converts it into TF-IDF vectors. TF-IDF gives higher importance to words that help distinguish one movie from another, while the n-gram setting also captures short two-word phrases. For a selected movie, cosine similarity is calculated between its vector and the other movie vectors. The highest-scoring different titles are returned.

2.3 Text Preprocessing

The notebook shows a preprocessing function that lowercases text, removes non-alphabetic characters, removes English stop words and lemmatizes the remaining words. The cleaned result is stored back in the tags column. This reduces superficial differences between words and creates a more consistent representation for TF-IDF.

2.4 Hybrid Application Logic

The application is best described as a hybrid product experience rather than a single hybrid machine-learning model. The core ML recommendation is TF-IDF similarity. Separately, TMDB is used to discover movies in the first genre associated with the selected title. The interface displays these as “Similar Movies (TF-IDF)” and “More Like This (Genre)”. The genre section is not learned by the TF-IDF model.

2.5 Main Modules

Module	Purpose
Data / model artifacts	Stores the processed dataframe, title-index mapping, TF-IDF vectorizer and TF-IDF matrix.
TF-IDF engine	Finds content-similar movie titles using cosine similarity.
TMDB service	Provides search, movie details, posters and genre discovery.

Module	Purpose
FastAPI backend	Loads model resources and exposes REST endpoints.
Streamlit frontend	Provides search, movie cards, details pages and recommendation sections.

Chapter 3: Dataset Description and Preprocessing

3.1 Data Source

The repository contains `movies_metadata.csv` and the notebook used for preparation and model building. The project also stores a processed dataframe as `df.pkl`. The notebook output shows a final processed dataframe/indexing stage containing 45,447 movie records, with titles mapped to row indices from 0 through 45,446.

3.2 Features Used by the Recommendation Pipeline

The notebook builds a consolidated text representation named `tags`. The project therefore does not use a conventional numeric target column. Movie title is the lookup key, while the text representation is the main input to TF-IDF. The application separately uses title, overview, release date, genres and poster-related fields when displaying TMDb movie information.

3.3 Preprocessing and Cleaning

Item	Observed implementation
Processed records	45,447 records shown in notebook output
Text field	<code>tags</code>
Lowercasing	Applied
Punctuation / non-letter removal	Applied with a regular expression
Stop-word removal	English stop words removed
Lemmatization	WordNetLemmatizer used in notebook
Duplicate title index	Title-to-index Series created with duplicates dropped
Vectorization	TfidfVectorizer
Maximum features	50,000
N-grams	(1, 2), unigrams + bigrams
Stop words in vectorizer	English

Table 1. Dataset and preprocessing summary.

3.4 Encoding and Scaling

Traditional categorical encoding is not required for the core recommendation calculation because the selected representation is text-based. Numeric feature scaling is also not performed as a separate preprocessing step. TF-IDF produces normalized text-weight vectors suitable for cosine similarity.

3.5 Train-Test Split and Target Variable

A train-test split is not present because the task is not framed as supervised prediction. There is no target label such as “recommended / not recommended” for each movie pair. Instead, the model creates a vector representation and ranks other movies by similarity. This is why classification metrics are not reported in Chapter 5.

3.6 Reproducibility Note

The preprocessing notebook uses NLTK resources for stop-word removal and lemmatization, while the `current_requirements.txt` contains the runtime dependencies for the FastAPI/Streamlit application but does not list NLTK. This is consistent with preprocessing being performed offline and the processed artifacts being saved as pickle files. If the notebook is rerun from scratch on another machine, its NLTK dependencies/resources should be documented separately.

Chapter 4: Implementation Details

4.1 Development Tools and Libraries

The repository uses Python as the main language. The runtime dependency file specifies FastAPI, Uvicorn, python-dotenv, httpx, pandas, NumPy, SciPy, scikit-learn and Streamlit. The notebook additionally uses the text-processing components required for NLTK preprocessing. TMDb is accessed through HTTP requests from the FastAPI backend.

Component	Technology / Library	Role
Machine learning	scikit-learn	TF-IDF vectorization
Similarity	NumPy / SciPy sparse operations	Vector similarity and ranking
Data handling	pandas	Movie dataframe processing
Backend	FastAPI + Uvicorn	REST API and model serving
Frontend	Streamlit	Interactive movie interface
External data	TMDb API	Search, details, posters and genre discovery
HTTP client	httpx / requests	Backend and frontend API communication
Serialization	pickle	Saved model/data artifacts

Table 2. Core implementation components.

4.2 TF-IDF Configuration

The notebook explicitly configures the vectorizer as follows:

```

tfidf = TfidfVectorizer(
    max_features=50000,
    ngram_range=(1, 2),
    stop_words='english'
)
```

The resulting sparse matrix is reported with shape (45447, 50000) and approximately 1.55 million stored elements. Sparse storage is appropriate because most vocabulary terms do not occur in most movie documents.

4.3 Recommendation Procedure

```

idx = indices[title]
sim_score = cosine_similarity(tfidi_matrix[idx], tfidi_matrix)
similar_idx = sim_score.argsort()[0][::-1][1:n+1]
return df['title'].iloc[similar_idx]
```

The deployed FastAPI implementation uses an equivalent sparse-matrix approach. It calculates the query row against the full TF-IDF matrix, sorts scores in descending order, skips the selected movie itself and returns the highest-ranked titles with similarity scores.

4.4 Backend API Design

The backend exposes health, home-feed, TMDb search, movie-detail, genre recommendation, TF-IDF recommendation and combined movie-search endpoints. The combined endpoint can return movie details plus TF-IDF and genre recommendations in one response. It also attempts to attach TMDb poster information to locally generated TF-IDF recommendations.

4.5 Frontend Implementation

The Streamlit application provides a home feed with trending, popular, top-rated, now-playing and upcoming categories. Users can type a movie keyword, see matching suggestions, open a movie-details view and then browse two recommendation groups. Session state and query parameters are used to preserve the selected movie and page state.

4.6 Saved Artifacts

Four important serialized files are included: `df.pkl`, `indices.pkl`, `tfidf.pkl` and `tfidf_matrix.pkl`. This moves the expensive preparation step out of normal application requests. At startup, FastAPI loads these artifacts and builds a normalized title-to-index mapping.

Chapter 5: Results and Analysis

5.1 What Was Actually Evaluated

The repository demonstrates recommendation outputs directly in the notebook. These examples provide a basic qualitative check that the similarity engine is producing related titles. However, the project does not contain a labeled relevance dataset or a formal offline evaluation such as Precision@K, Recall@K or NDCG. Therefore, this report does not invent accuracy-style scores.

Input movie	Example recommendations observed in notebook	Interpretation
Avatar	Avatar 2; The Inhabited Island; Thor: Ragnarok; Moontrap; The Edge of Hell; The Tenth Muse; The Last Days of Pompeii	The first three are related to the content signals; Avatar 2 and The Inhabited Island are related to the plot signals; Thor: Ragnarok is related to the genre signals; Moontrap is related to the director signals; The Edge of Hell is related to the cast signals; The Tenth Muse is related to the crew signals; The Last Days of Pompeii is related to the cast signals.
The Dark Knight Rises	Batman Forever; The Dark Knight; Batman: Mask of the Phantasm; Batman Returns; Batman: The Movie - Adam West	Strong franchise and character relationships are visible.
Toy Story	Toy Story 2; Toy Story 3; Small Fry; Superstar Goofy; Grover's First Movie	The first results are sensible, while lower-ranked results are noisy.

Table 3. Recommendation examples observed in the project notebook.

5.2 Why Accuracy, Precision, Recall and F1 Are Not Reported

Accuracy, precision, recall, F1-score, ROC-AUC and a classification confusion matrix require labeled outcomes against which predictions can be compared. This project does not predict a class; it ranks movies by similarity. Reporting those metrics here would create a false sense of model performance. A more appropriate future evaluation would use recommendation-specific metrics, human relevance judgments or a held-out interaction dataset.

5.3 Strengths of the Current System

- The TF-IDF pipeline is simple, explainable and relatively easy to reproduce.
- Using bigrams in addition to unigrams gives the representation some ability to capture short phrases.
- Precomputed sparse artifacts make runtime recommendation much lighter than retraining for every request.
- FastAPI separates the recommendation engine from the Streamlit interface.
- TMDB integration adds current search, posters, movie details and genre discovery.
- The interface exposes both content similarity and genre discovery.

5.4 Limitations

- Content similarity depends on the quality and completeness of the movie text.
- The system is not personalized; users selecting the same movie receive the same local ranking.
- Cosine similarity can surface similar wording without necessarily matching a user's taste.
- Genre recommendations use the first genre returned by TMDB, simplifying multi-genre movies.
- The Toy Story example shows that lower-ranked results can become noisy.
- Formal recommendation metrics and user studies are not included yet.

5.5 Overall Result

As an end-to-end student ML project, the strongest result is not a single numerical score but the successful connection of preprocessing, vectorization, similarity ranking, model persistence, API serving and a usable frontend. The examples show that the engine can recover obvious relationships such as franchise and character similarity. It is therefore a credible content-based baseline, with clear room for more rigorous evaluation and personalization.

Chapter 6: Conclusion

6.1 Conclusion

The Movie Recommendation System demonstrates a complete path from movie metadata to an interactive recommendation application. The machine-learning core uses TF-IDF to convert movie text into numerical vectors and cosine similarity to rank related titles. The preprocessing stage makes the text more consistent through normalization, stop-word removal and lemmatization. The resulting resources are serialized and loaded by a FastAPI backend, which is then consumed by a Streamlit interface.

A useful part of the project is the separation between the local recommendation engine and live TMDB services. The local artifacts provide stable similarity calculations, while TMDB supplies user-facing movie search and visual metadata. The result is more than a standalone notebook: it is a small full-stack ML application with a clear model-serving layer.

6.2 Future Improvements

- Add collaborative filtering using user ratings or watch-history data.
- Introduce personalized recommendations so different users receive different rankings.
- Evaluate recommendations using Precision@K, Recall@K, MAP or NDCG on an appropriate interaction dataset.
- Improve ranking by combining TF-IDF similarity, multiple genres, popularity and ratings.
- Use richer semantic embeddings to capture meaning beyond exact vocabulary overlap.
- Add filtering by release year, language, rating, genre and availability.
- Add user authentication, watchlists and feedback so the system can learn from user actions.
- Package the application with Docker and add automated deployment/CI checks.
- Document notebook-only dependencies such as NLTK separately from runtime dependencies.

6.3 Final Assessment

The project is a strong practical baseline for demonstrating text preprocessing, feature extraction, similarity-based recommendation, model persistence, API development and UI integration. Its next major step should be evaluation and personalization rather than simply increasing the TF-IDF matrix size. That would turn the current content-similarity application into a more measurable recommendation product.

References / Bibliography

1. Project Repository

DhananiParita, Movie-Recommendation-System, GitHub repository, accessed October 2026.

2. Scikit-learn Documentation

Documentation for TfidfVectorizer and cosine similarity, used as the technical basis for the vectorization and similarity approach.

3. The Movie Database (TMDB) API

TMDB API documentation, used for movie search, details, posters and genre-based discovery.

4. FastAPI Documentation

FastAPI documentation, used for the REST backend.

5. Streamlit Documentation

Streamlit documentation, used for the interactive frontend.

6. Supplied ML Project Report Format

The supplied format specifies the title page, 150–250 word abstract, table of contents, six chapters and references used in this report. `filecite{turn0file0{L3-L27}`

Source-grounding note

This report is based on the repository files README.md, movies.ipynb, main.py, app.py and requirements.txt. Where the repository does not provide supervised evaluation metrics, the report explicitly says so rather than inventing scores. No fabricated screenshots or model results have been added.